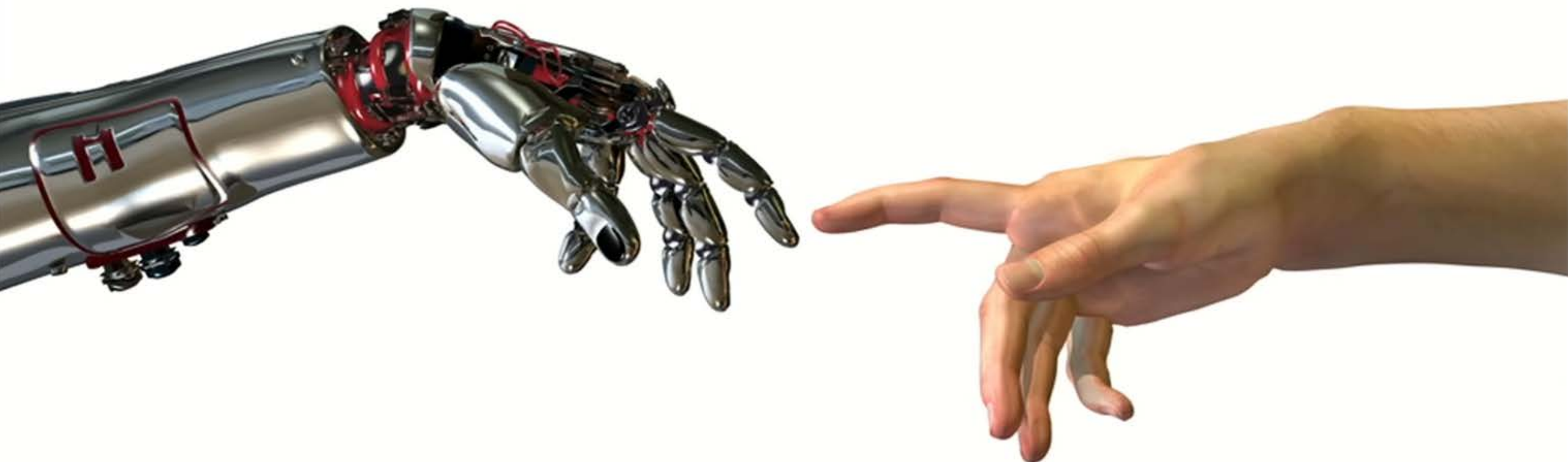


06026240 INTELLIGENT SYSTEM DEVELOPMENT

Chapter VI Deep Reinforcement Learning

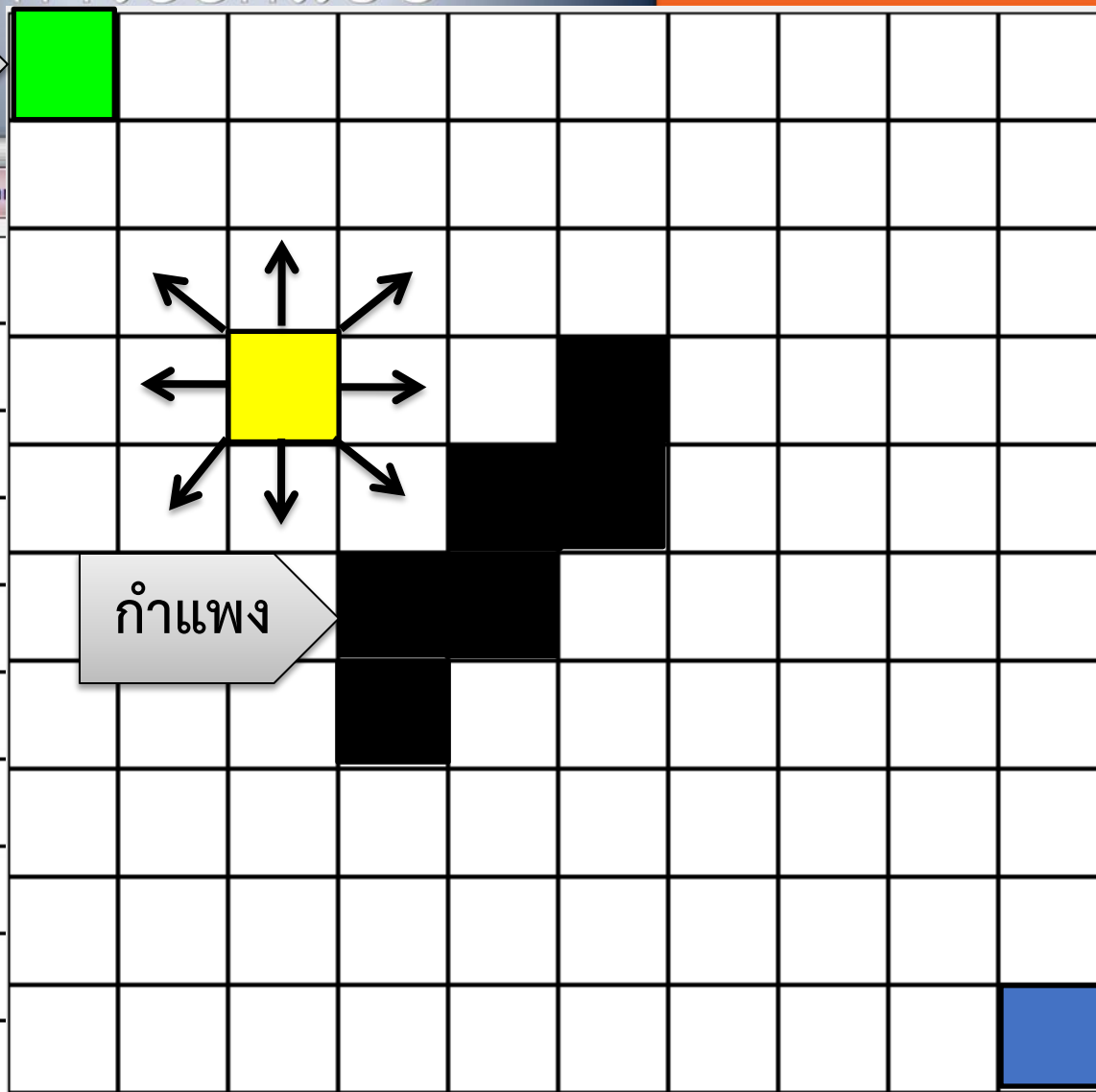
with Dr. Bhattarabhorn Wattanacheep

Learning in Dynamic Environments



การออกแบบ

จุดเริ่มต้น



กำแพง

จุดสุดท้าย

Reinforcement Learning: Robots, Games, the World

Robotics



Game Play and Strategy



Classes of Learning Problems

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn function to map
 $x \rightarrow y$

Apple example:



This thing is an apple.

Unsupervised Learning

Data: x

x is data, no labels!

Goal: Learn underlying
structure

Apple example:



This thing is like
the other thing.

Reinforcement Learning

Data: state-action pairs

Goal: Maximize future rewards
over many time steps

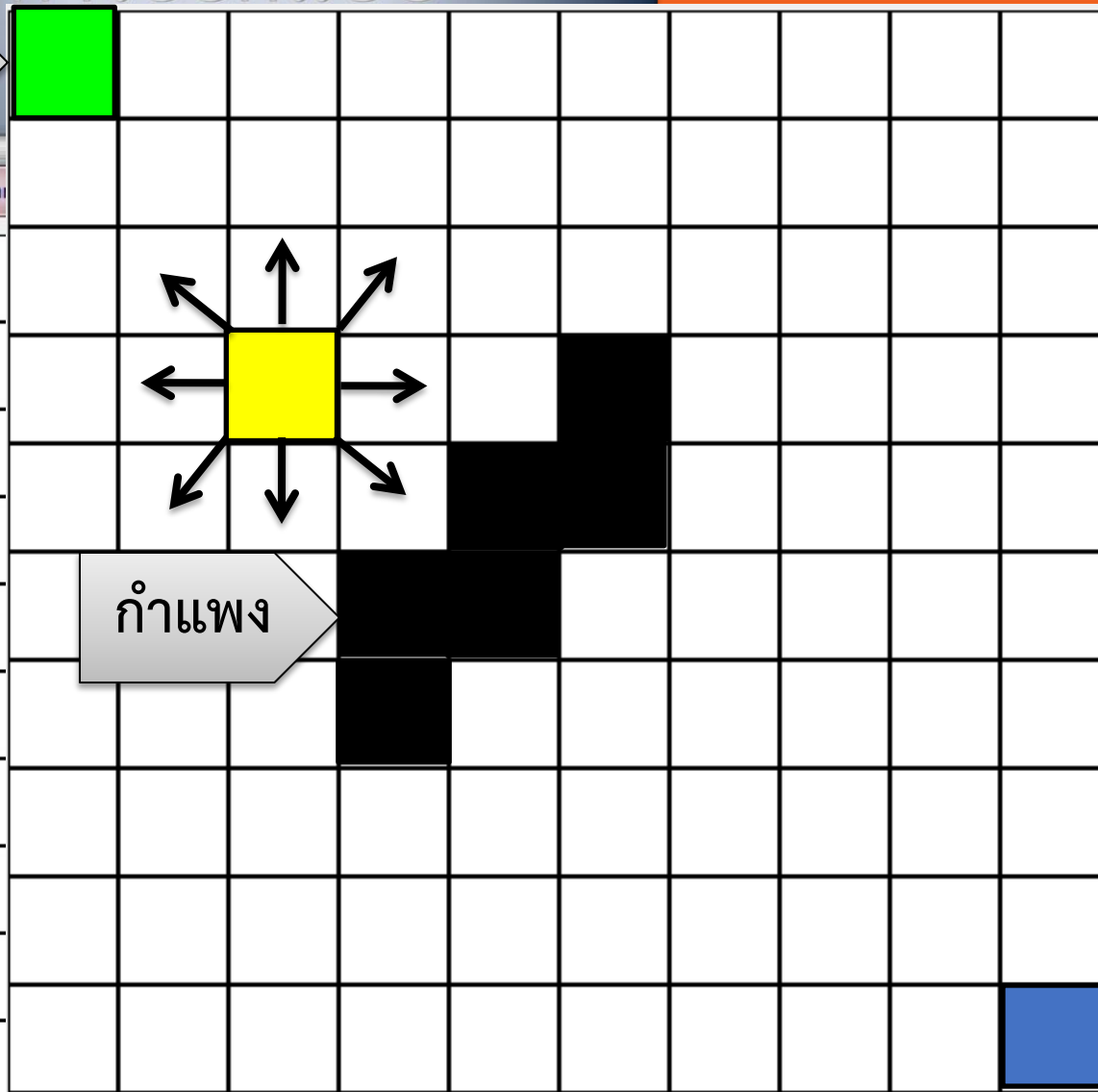
Apple example:



Eat this thing because it
will keep you alive.

การออกแบบ

จุดเริ่มต้น



กำแพง

จุดสุดท้าย

Reinforcement Learning (RL): Key Concepts



AGENT

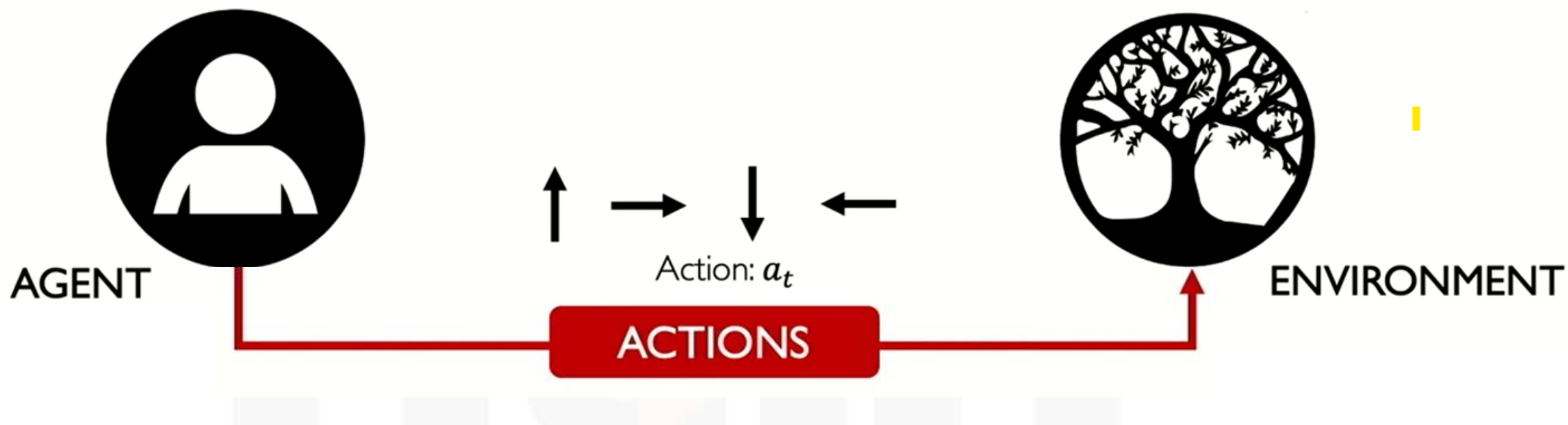


ENVIRONMENT

Agent: takes actions.

Environment: the world in which the agent exists and operates.

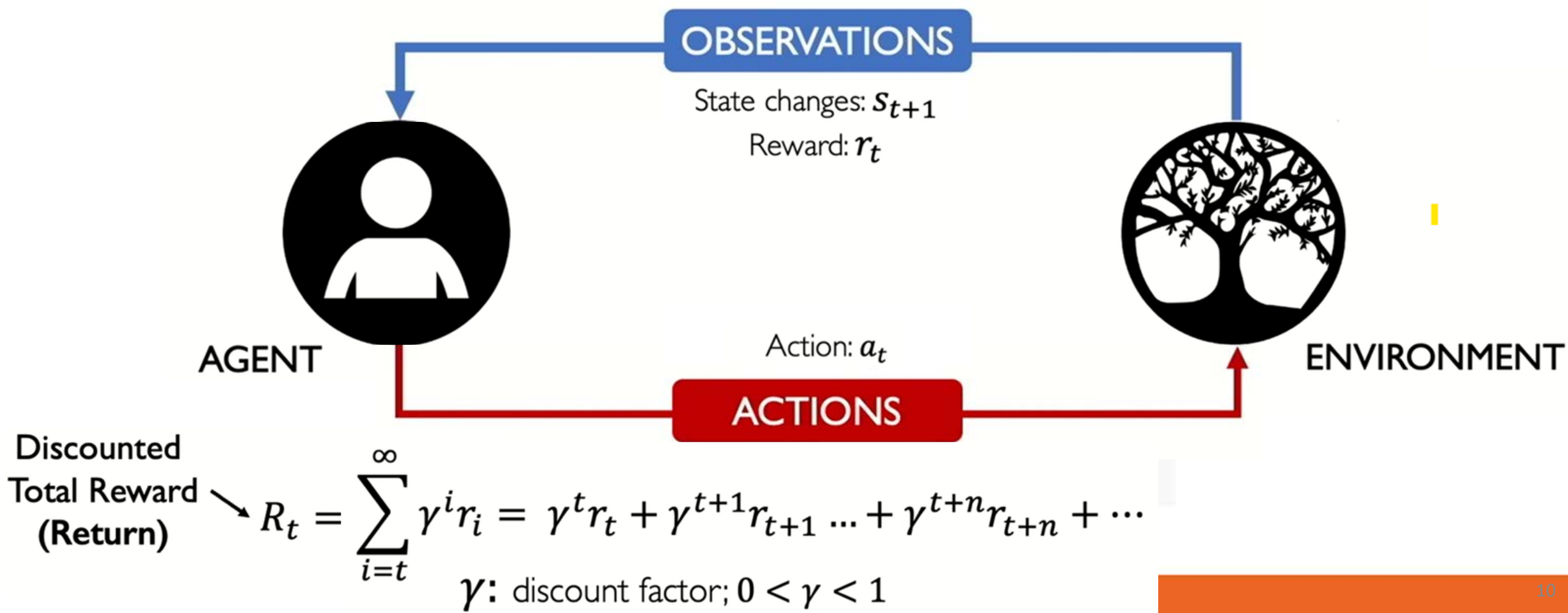
Reinforcement Learning (RL): Key Concepts



Action: a move the agent can make in the environment.

Action space A : the set of possible actions an agent can make in the environment

Reinforcement Learning (RL): Key Concepts



Return: ผลตอบแทนรวมแบบถ่วงลดจากเวลา t

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$$

r_t
รางวัลทันทีที่เวลา t

$r_{t+1}, r_{t+2}, \dots$
รางวัลในอนาคต

γ
ตัวลดน้ำหนักอนาคต

$\gamma^k r_{t+k}$
รางวัลอนาคตหลังลดค่า

R_t
ผลรวมทั้งหมด

- Return ไม่ใช่แค่รางวัลปัจจุบัน แต่รวมรางวัลอนาคตทั้งหมด
- รางวัลที่อยู่ไกลออกไปถูกคูณด้วย γ หลายครั้ง จึงมีน้ำหนักน้อยลง
- เป้าหมายของ RL คือเลือก action ที่ทำให้ expected return สูงที่สุด

Discount factor γ : ทำไมต้องลดค่าน้ำหนักของอนาคต

$$0 < \gamma < 1$$

γ ใกล้ 0

สนใจรางวัลระยะสั้นมาก

γ ประมาณ 0.8–0.95

สมดุลระยะสั้นและยาว

γ ใกล้ 1

มองอนาคตไกลขึ้น

ก้าวอนาคต k	$\gamma=0.5$	$\gamma=0.9$
0	1.000	1.000
1	0.500	0.900
2	0.250	0.810
3	0.125	0.729

ตารางแสดง “น้ำหนัก” γ^k ที่คูณกับ reward ในอนาคต

ตัวอย่างคำนวณ Return ที่ละขั้น

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2}$$

กำหนด reward
 $r_t=5, r_{t+1}=2, r_{t+2}=10$

กำหนด γ
 $\gamma = 0.9$

แทนค่า
 $5 + 0.9(2) + 0.9^2(10)$

ขั้นคำนวณ

$$R_t = 5 + 1.8 + 8.1 = 14.9$$

สรุป: action ที่ให้ reward ท้นทีน้อย อาจดีกว่า ถ้าทำให้ reward ในอนาคตสูงกว่า

Defining the Q-function

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Total reward, R_t , is the discounted sum of all rewards obtained from time t

$$Q(s_t, a_t) = \mathbb{E}[R_t | s_t, a_t]$$

The Q-function captures the **expected total future reward** an agent in **state, s** , can receive by executing a certain **action, a**

Q-function: ค่าคาดหวังของ Return เมื่อเริ่มจาก state และ action หนึ่ง

$$Q(s_t, a_t) = E[R_t | s_t, a_t]$$

s_t
สถานะปัจจุบัน

a_t
action ที่เลือก

R_t
return หลังจาก
นั้น

$E[]$
ค่าเฉลี่ยเชิงความ
น่าจะเป็น

- Q ไม่ใช่ reward ทันที แต่คือ “ผลตอบแทนรวมในอนาคตที่คาดว่าจะได้”
- เงื่อนไข | s_t, a_t แปลว่า เริ่มจาก state นี้ และเลือก action นี้ก่อน
- ถ้า environment มีความไม่แน่นอน ต้องเฉลี่ยทุกผลลัพธ์ที่เป็นไปได้

วิธีคำนวณ Q เมื่อผลลัพธ์มีหลายกรณี

$$Q(s,a) = \sum p_i \times R_i$$

Action A
70 % ได้ $R=4$
30 % ได้ $R=-5$

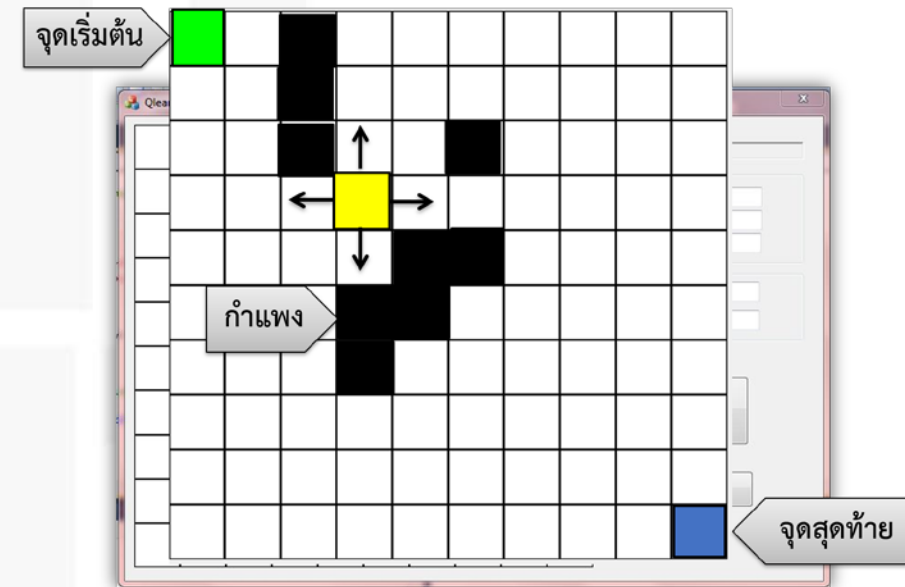
คำนวณ
 $Q(s,A) = 0.7(4) + 0.3(-5)$

ผลลัพธ์
 $Q(s,A) = 1.3$

Action B
100 % ได้ $R=4$

คำนวณ
 $Q(s,B) = 1.0(4)$

ผลลัพธ์
 $Q(s,B) = 4$



ถ้าเลือกตาม Q-value: เลือก Action A เพราะ $5.5 > 4$

เชื่อมกับ Bellman idea: คิดทีละก้าวได้

$$Q(s_t, a_t) = r_t + \gamma \max_a Q(s_{t+1}, a)$$

r_t
reward ทันที

s_{t+1}
state ถัดไป

$\max_a Q$
เลือก action ถัดไปที่ดีที่สุด

γ
ลดค่าอนาคต

- Bellman equation แยก “ตอนนี้” ออกจาก “อนาคต”
- เหมาะกับการเรียนรู้แบบ Q-learning: ปรับค่า Q จากประสบการณ์ซ้ำ ๆ
- ใน Deep Q-Network (DQN) ใช้ neural network ประมาณค่า $Q(s, a)$

How to take actions given a Q-function?

$$Q(\overset{\uparrow}{s_t}, \overset{\uparrow}{a_t}) = \mathbb{E}[R_t | s_t, a_t]$$

(state, action)

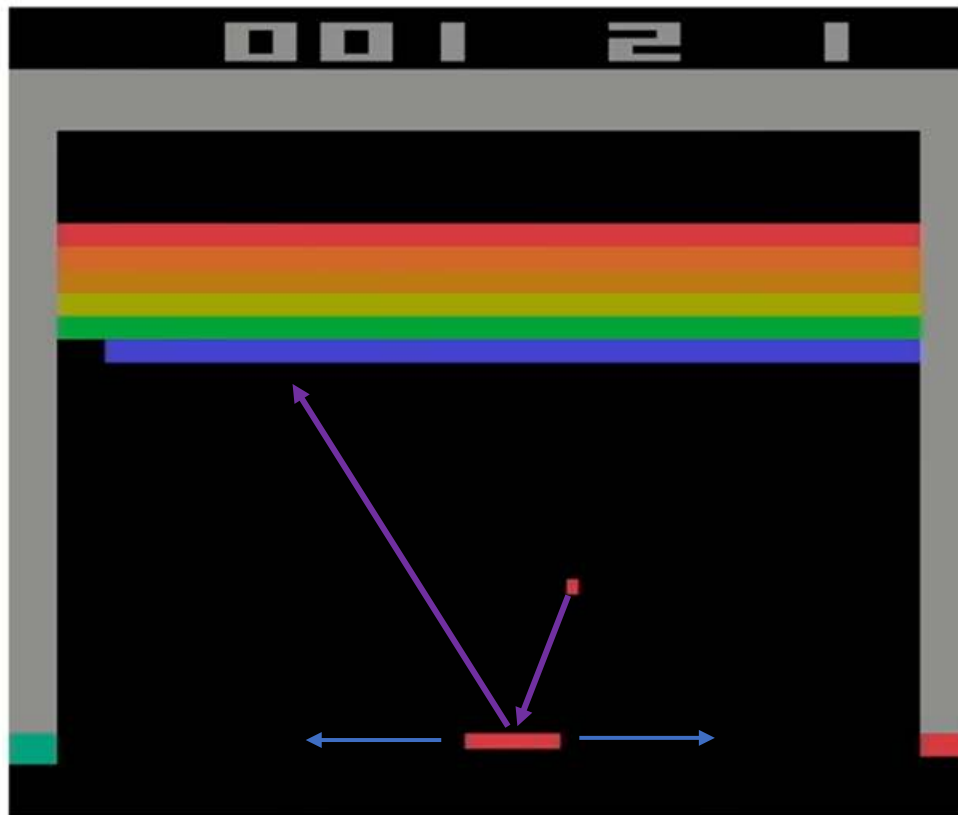
Ultimately, the agent needs a **policy** $\pi(s)$, to infer the **best action to take** at its state, s

Strategy: the policy should choose an action that maximizes future reward

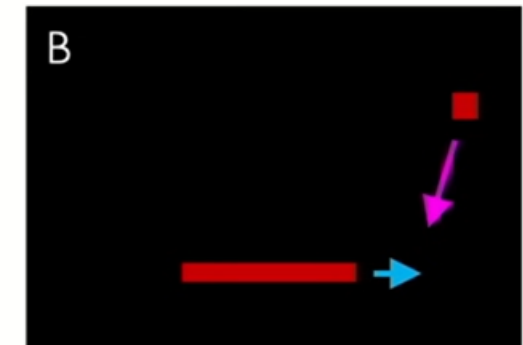
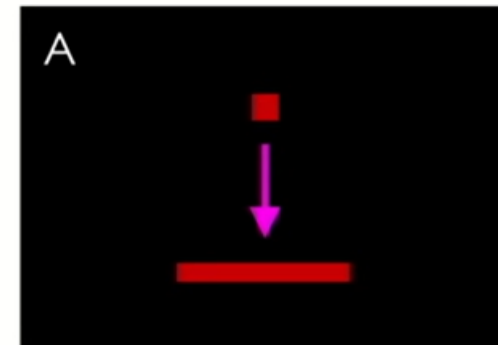
$$\pi^*(s) = \underset{a}{\operatorname{argmax}} Q(s, a)$$

Digging deeper into the Q-function

Example: Atari Breakout



It can be very difficult for humans to accurately estimate Q-values



Which (s, a) pair has a higher Q-value?

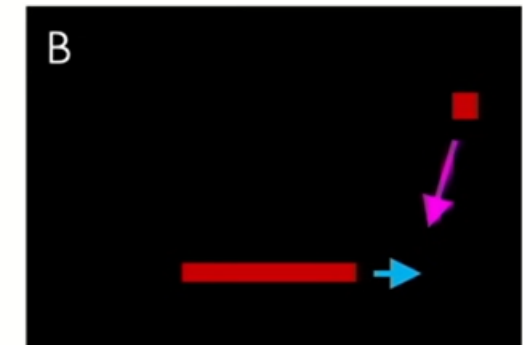
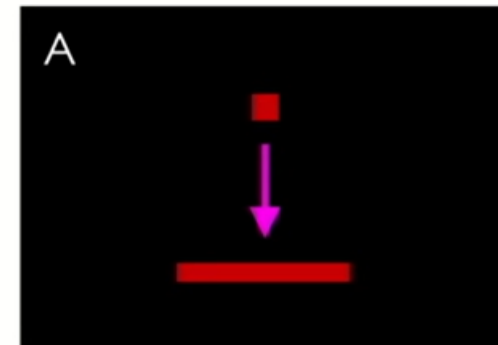


Digging deeper into the Q-function

Example: Atari Breakout



It can be very difficult for humans to accurately estimate Q-values



Which (s, a) pair has a higher Q-value?



Deep Reinforcement Learning Algorithms

Value Learning

Find $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

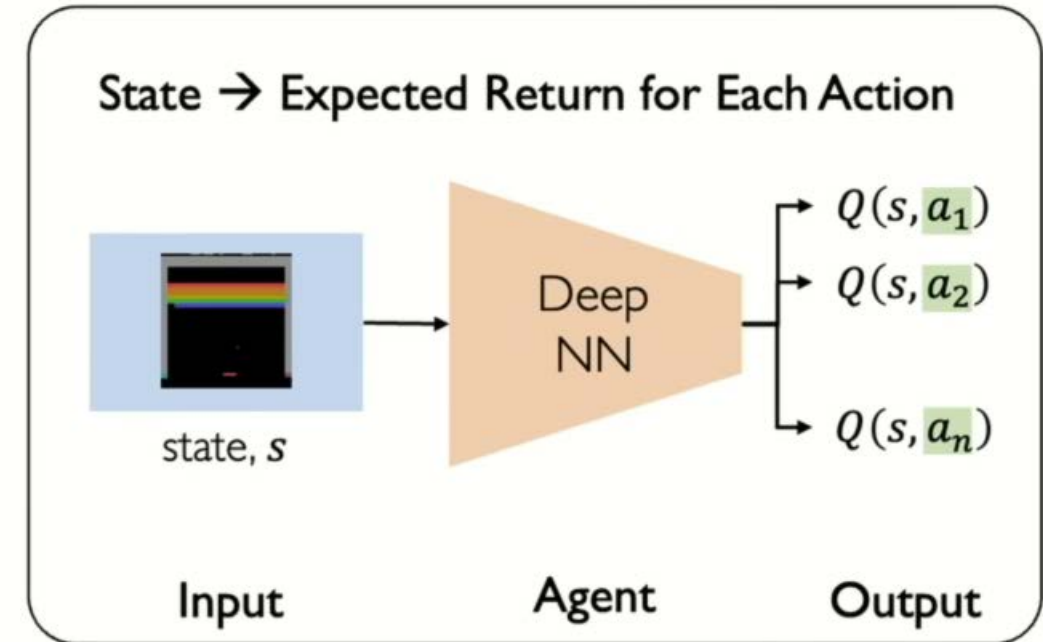
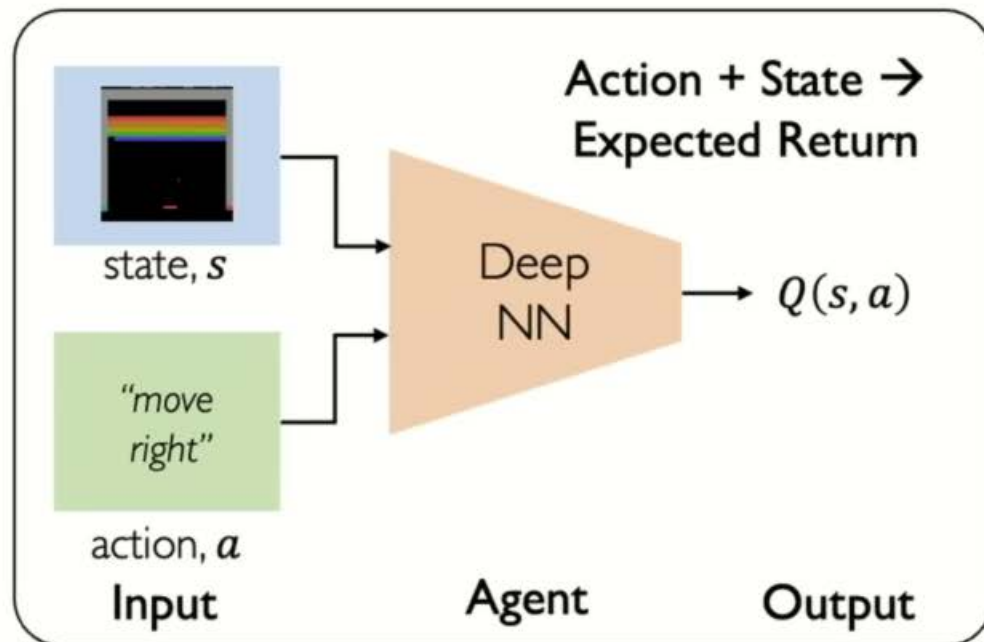
Policy Learning

Find $\pi(s)$

Sample $a \sim \pi(s)$

Deep Q Networks (DQN)

How can we use deep neural networks to model Q-functions?

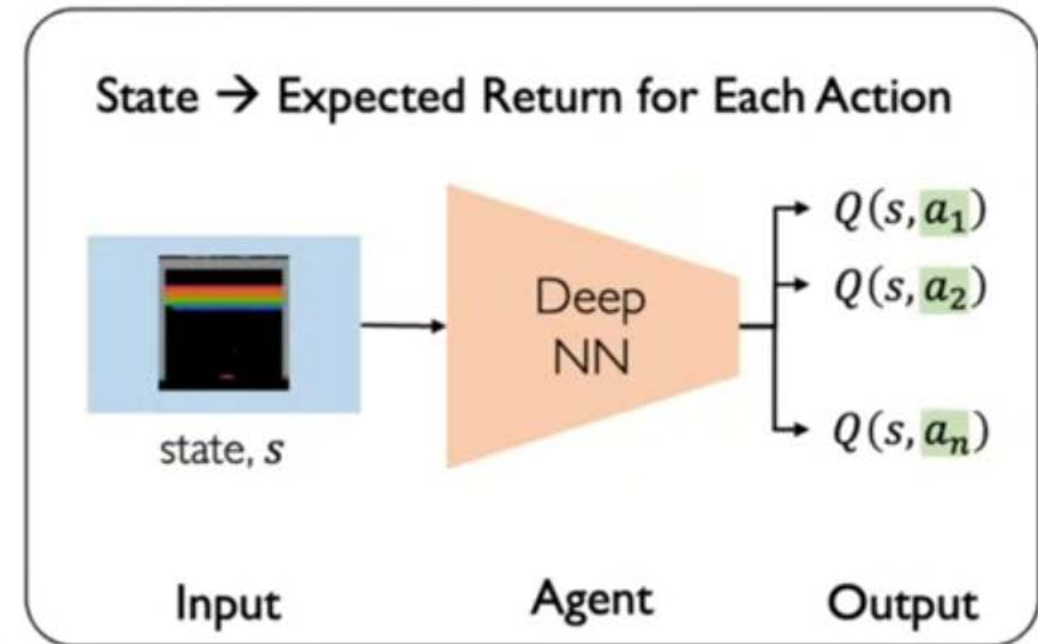
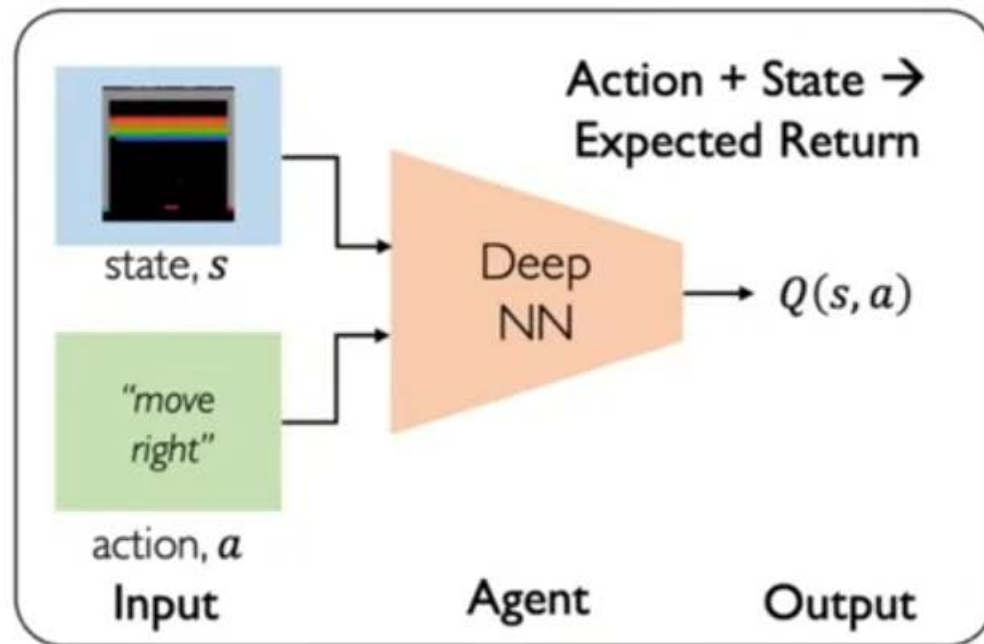


$$\pi^* = \arg \max_a Q(s, a)$$

หมายเหตุ: ในช่วงฝึก อาจไม่ได้เลือก Q สูงสุดเสมอ เพราะต้องมี exploration เช่น ϵ -greedy

Deep Q Networks (DQN): Training

How can we use deep neural networks to model Q-functions?



target

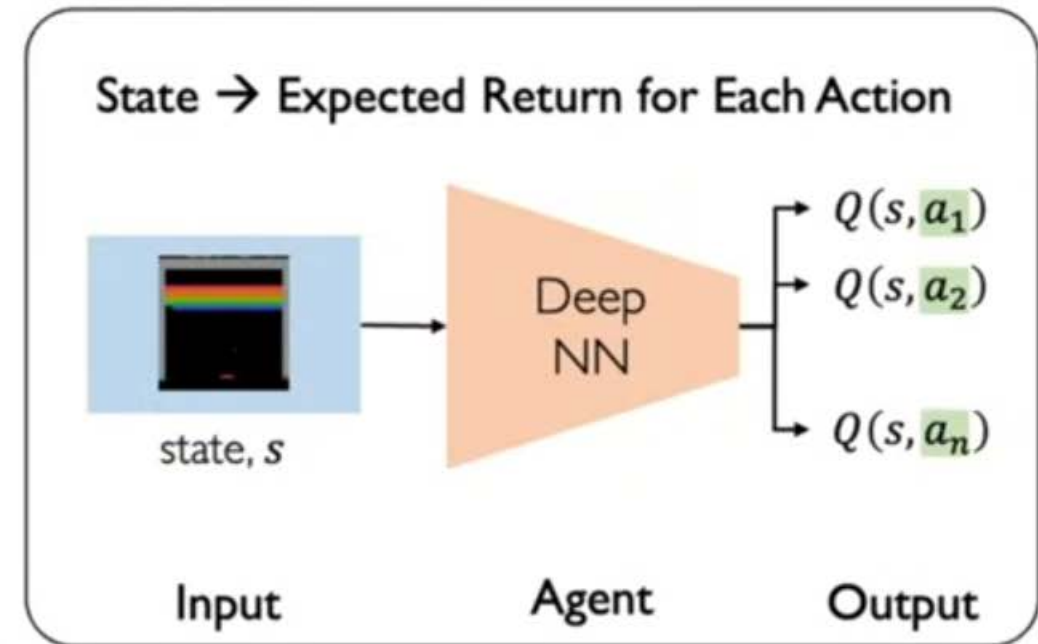
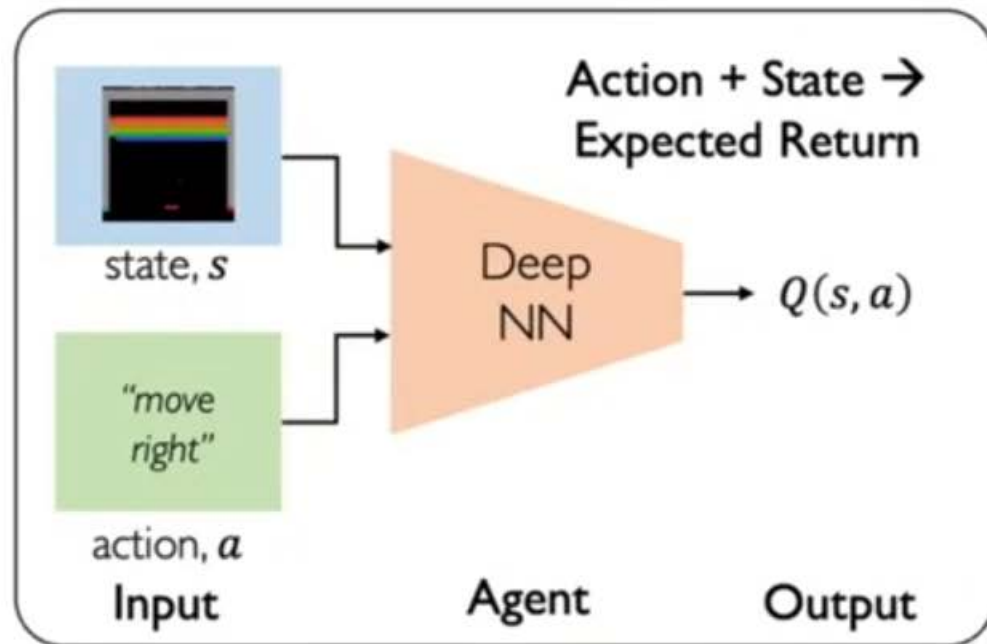
$$\left(r + \gamma \max_{a'} Q(s', a') \right)$$



Take all the best actions $\rightarrow$ target return

Deep Q Networks (DQN): Training

How can we use deep neural networks to model Q-functions?

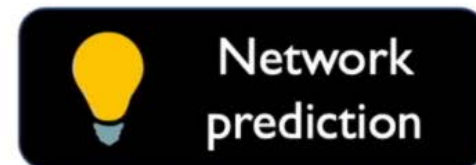


target

$$\left(r + \gamma \max_{a'} Q(s', a') \right)$$

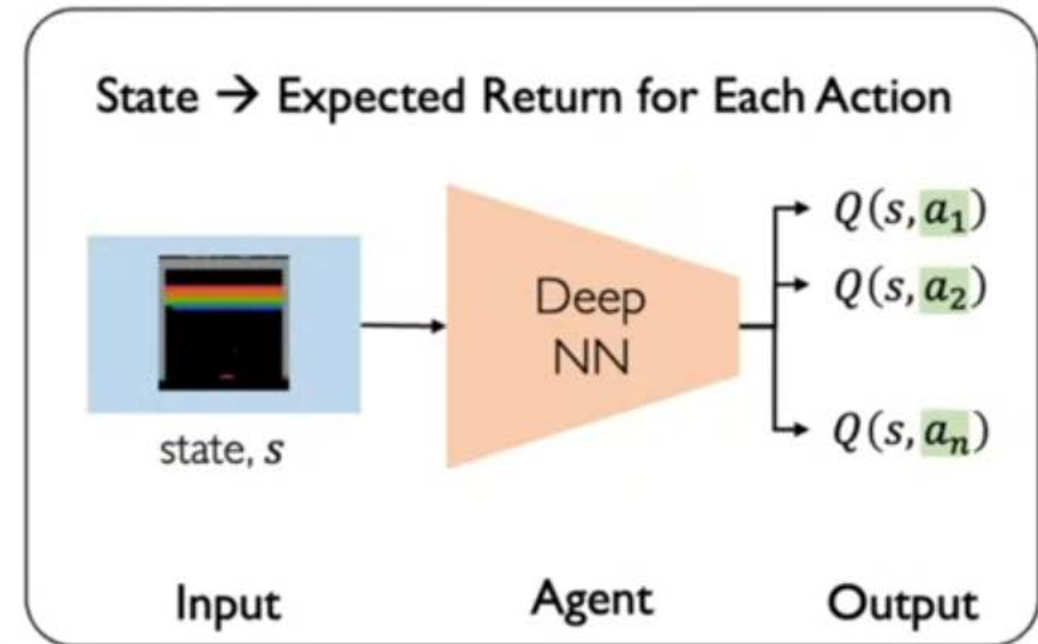
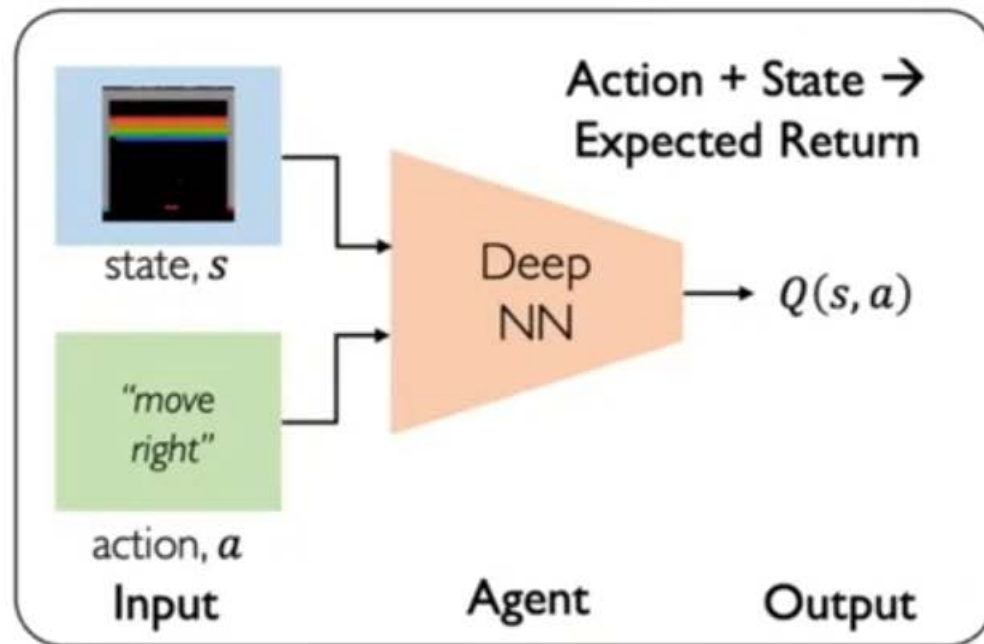
predicted

$$Q(s, a)$$



Deep Q Networks (DQN): Training

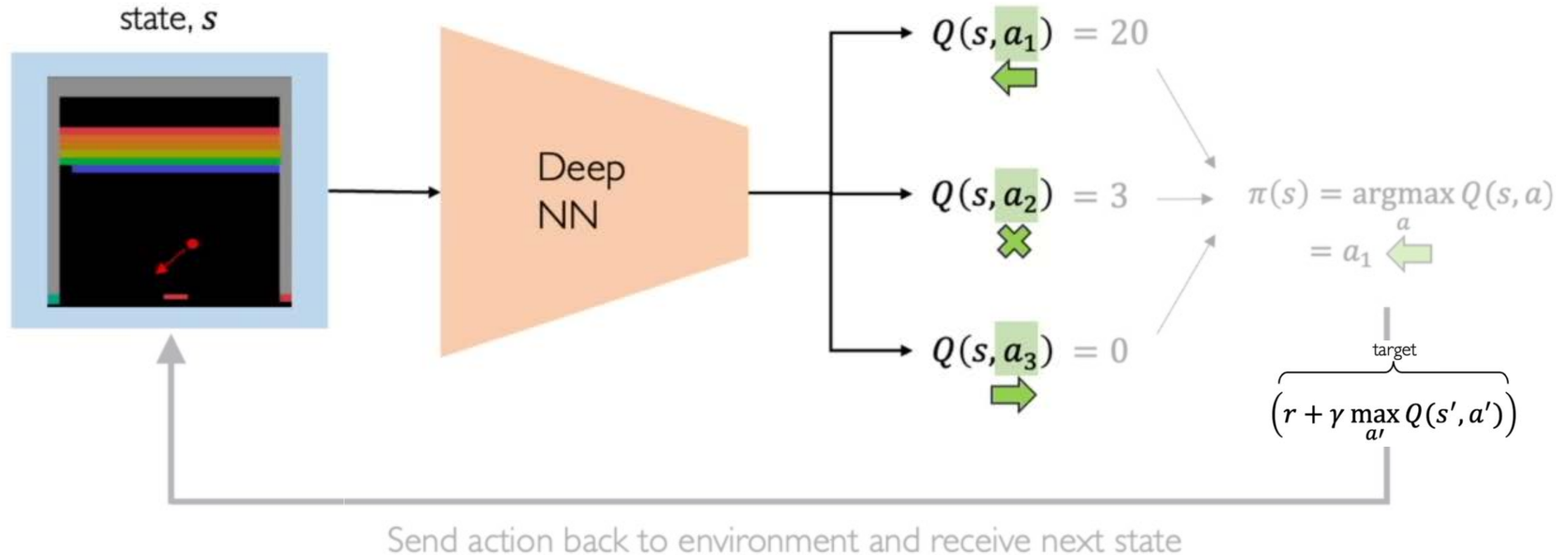
How can we use deep neural networks to model Q-functions?



$$\mathcal{L} = \mathbb{E} \left[\left\| \underbrace{\left(r + \gamma \max_{a'} Q(s', a') \right)}_{\text{target}} - \underbrace{Q(s, a)}_{\text{predicted}} \right\|^2 \right] \quad \text{Q-Loss}$$

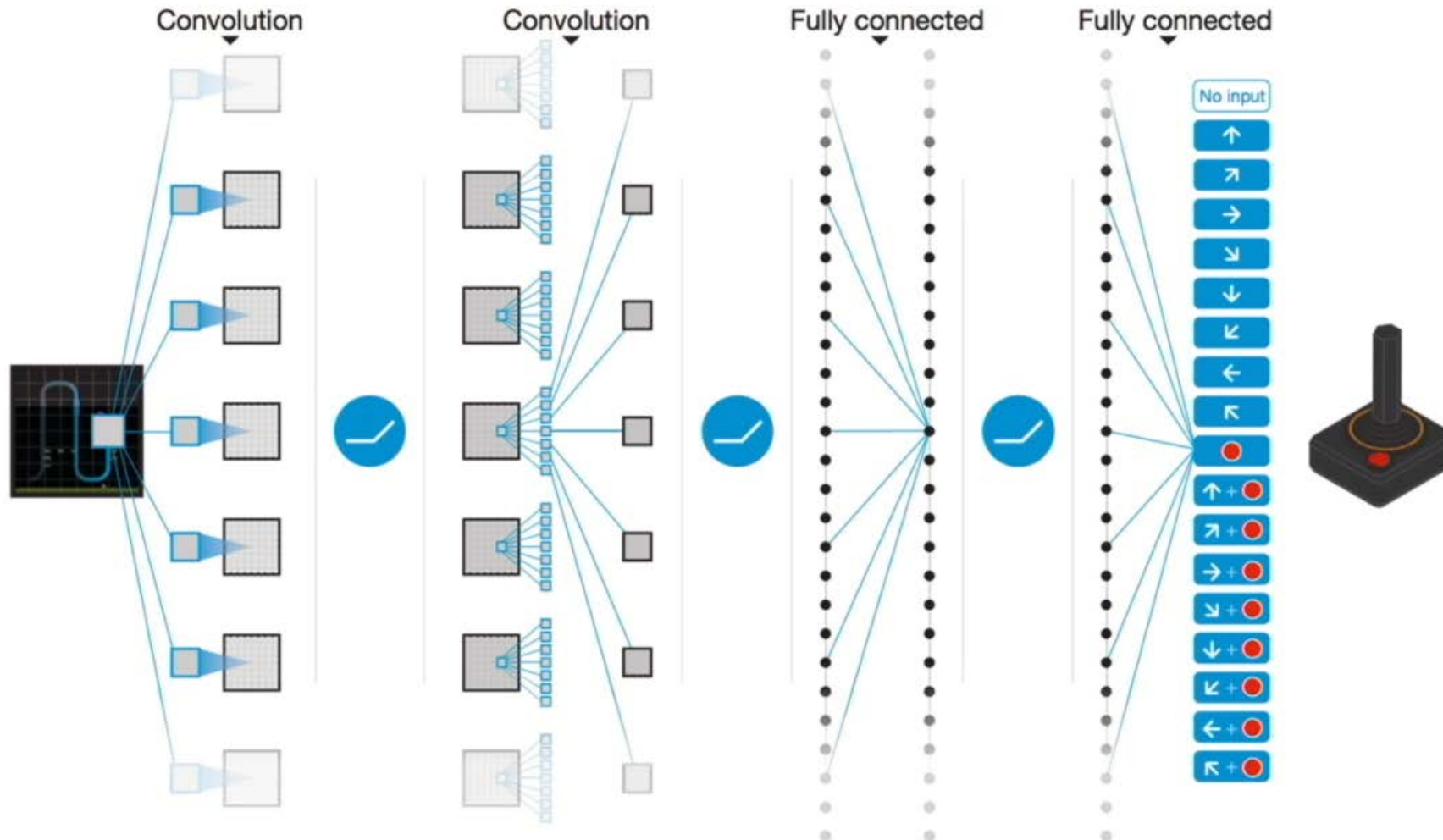
Deep Q Network Summary

Use NN to learn Q-function and then use to infer the optimal policy, $\pi(s)$

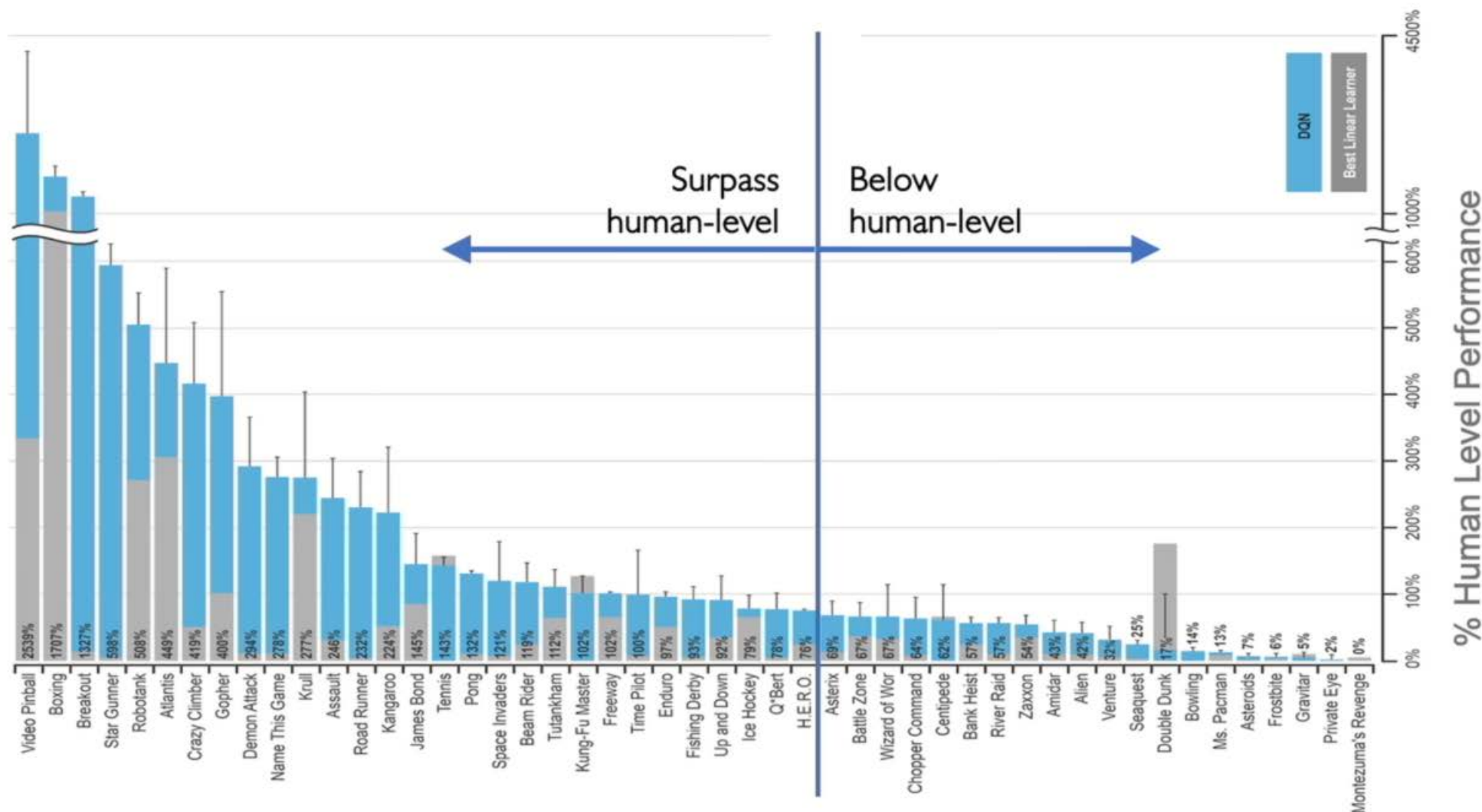


หมายเหตุ: ในช่วงฝึก อาจไม่ได้เลือก Q สูงสุดเสมอ เพราะต้องมี exploration เช่น ϵ -greedy

DQN Atari Results



DQN Atari Results



Downsides of Q-learning

Complexity:

- Can model scenarios where the action space is discrete and small
- Cannot handle continuous action spaces

Flexibility:

- Policy is deterministically computed from the Q function by maximizing the reward → cannot learn stochastic policies

Deep Reinforcement Learning Algorithms

Value Learning

Find $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

Policy Learning

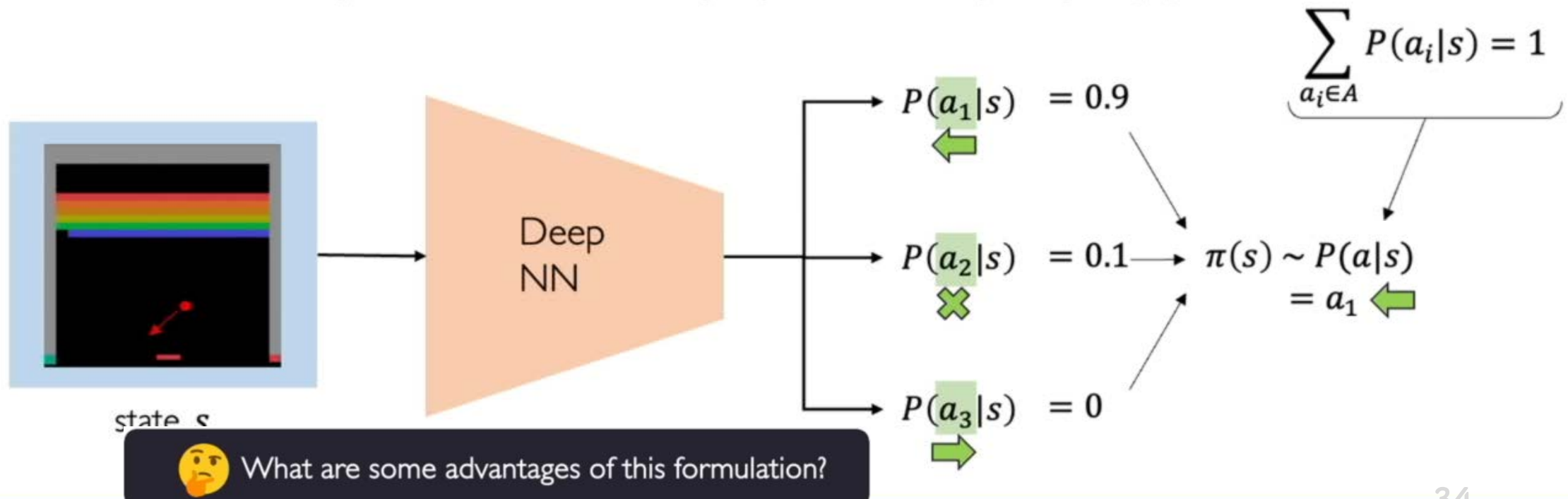
Find $\pi(s)$

Sample $a \sim \pi(s)$

Policy Gradient (PG): Key Idea

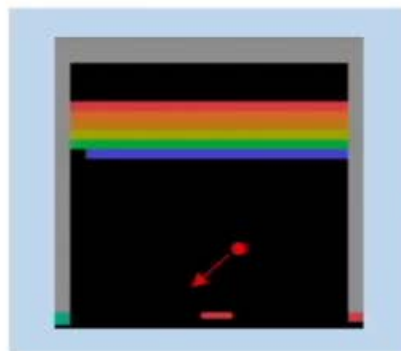
DQN: Approximate Q-function and use to infer the optimal policy, $\pi(s)$

Policy Gradient: Directly optimize the policy $\pi(s)$

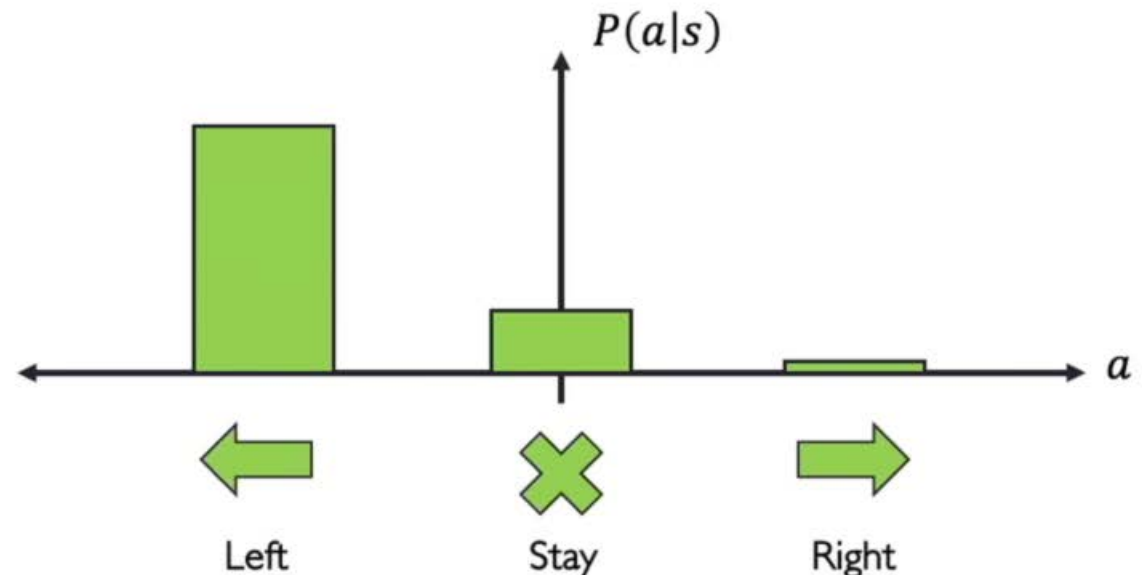


Discrete vs Continuous Action Spaces

Discrete action space: which direction should I move?



state, s

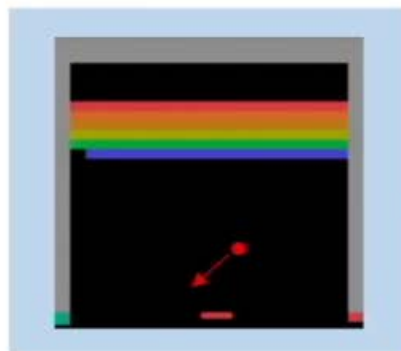


Discrete vs Continuous Action Spaces

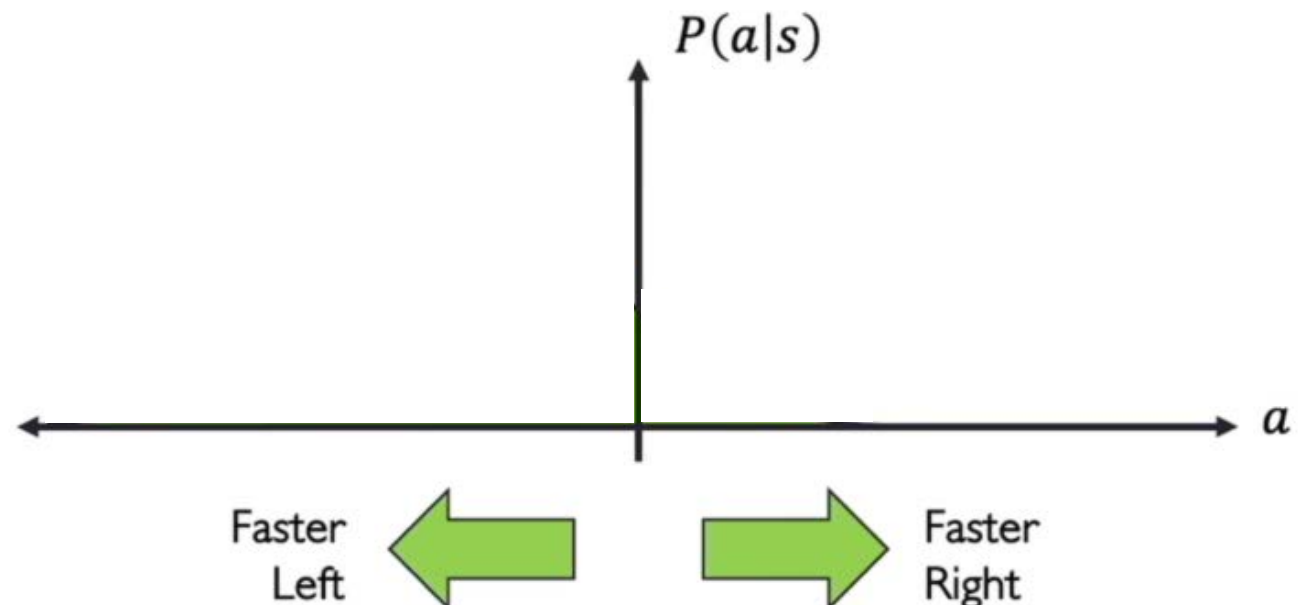
Discrete action space: which direction should I move?



Continuous action space: how fast should I move?

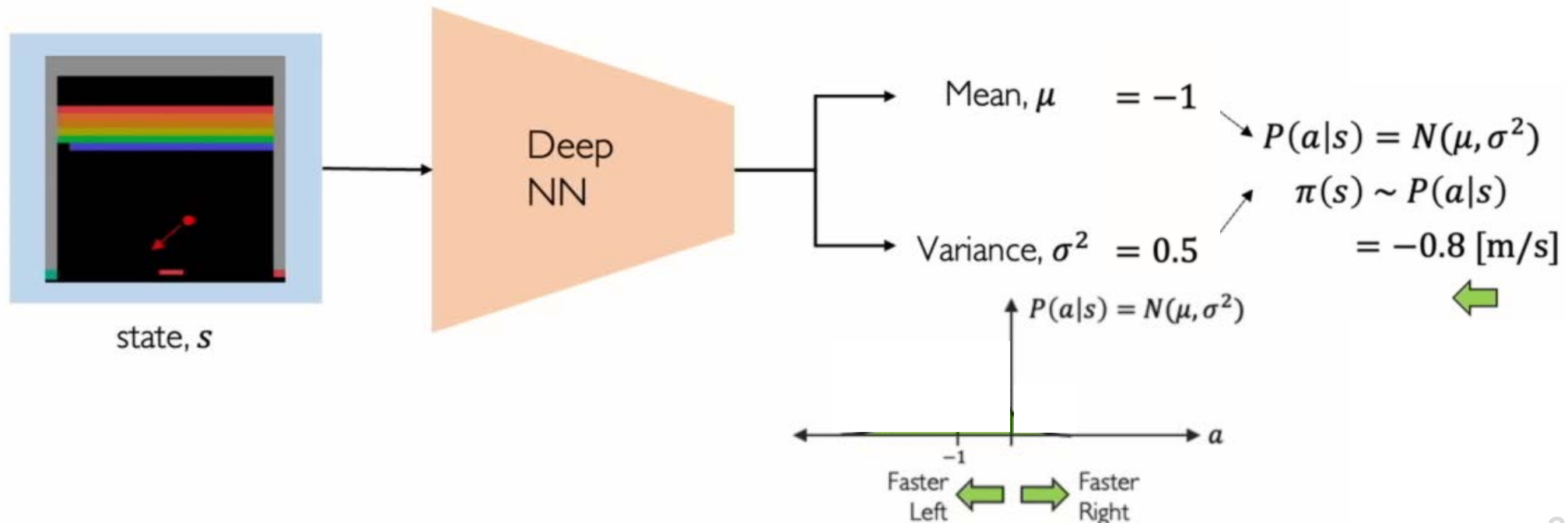


state, s



Policy Gradient (PG): Key Idea

Policy Gradient: Enables modeling of continuous action space



Training Policy Gradients: Case Study

Reinforcement Learning Loop:



Case Study – Self-Driving Cars

Agent: vehicle

State: camera, lidar, etc

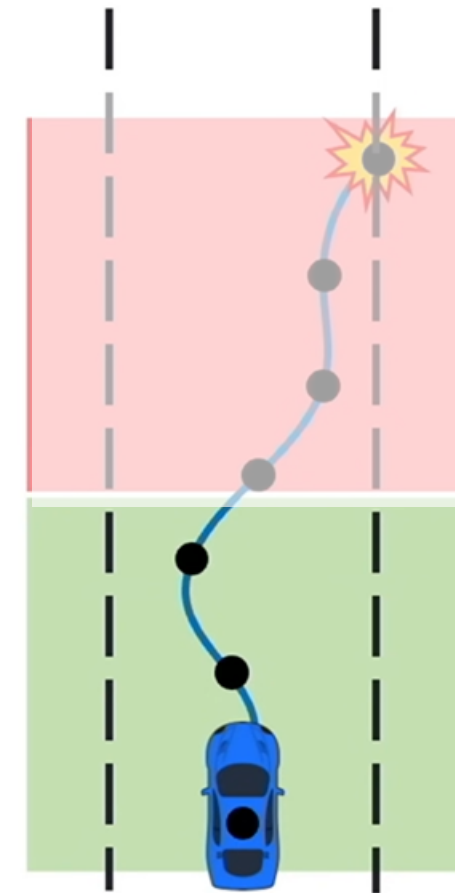
Action: steering wheel angle

Reward: distance traveled

Training Policy Gradients

Training Algorithm

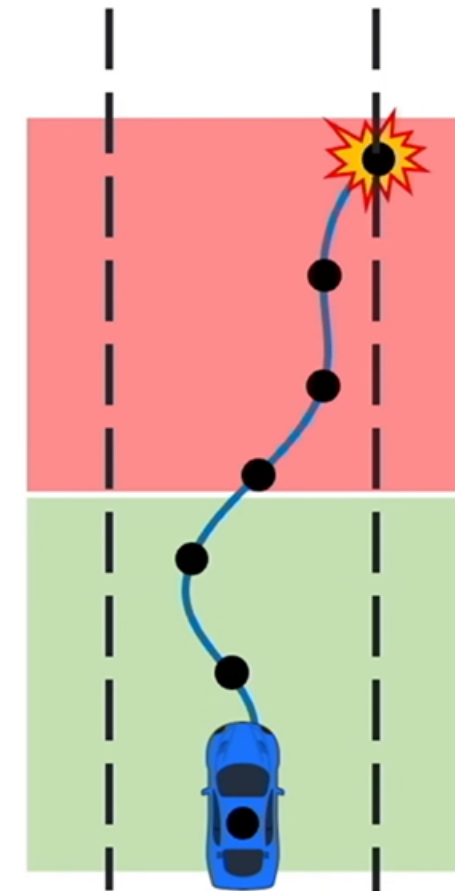
1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward



Training Policy Gradients

Training Algorithm

1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward



Training Policy Gradients

Training Algorithm

1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward



Training Policy Gradients

Training Algorithm

1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward

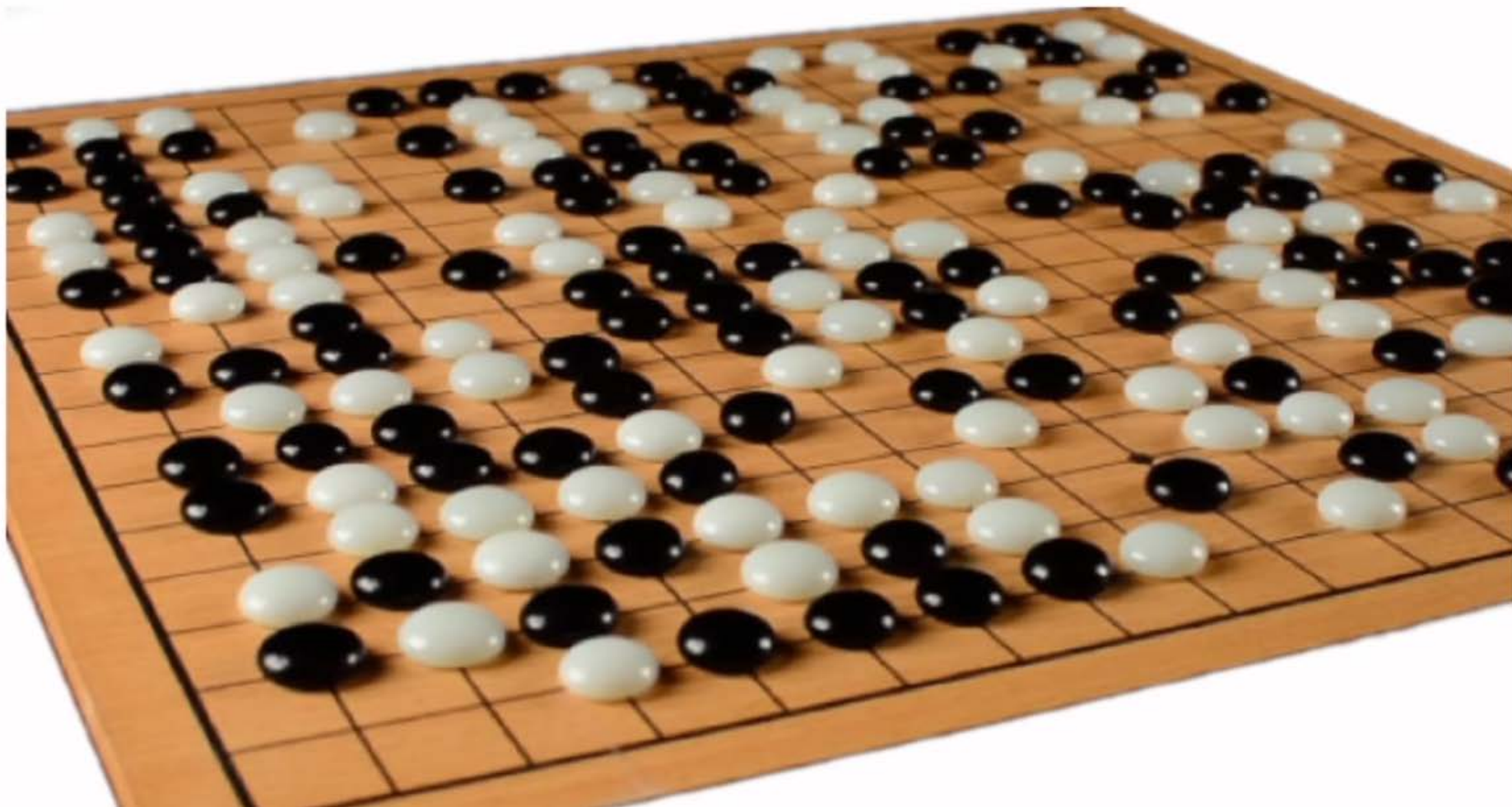
log-likelihood of action

$$\text{loss} = -\log P(a_t | s_t) R_t$$

reward

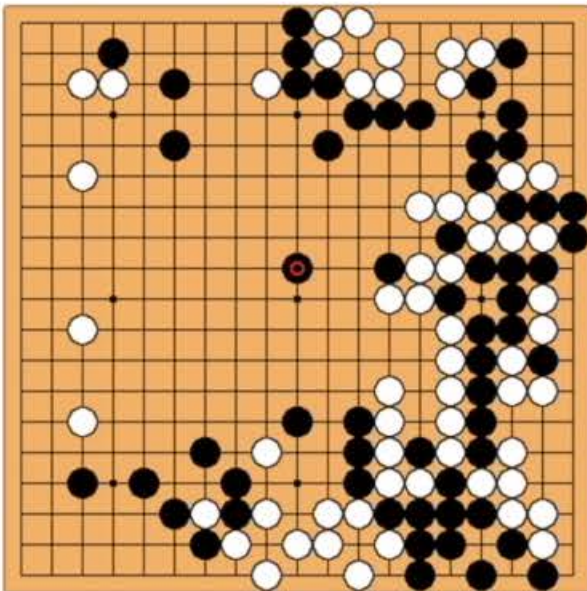
Deep Reinforcement Learning Applications

Reinforcement Learning and the Game of Go



The Game of Go

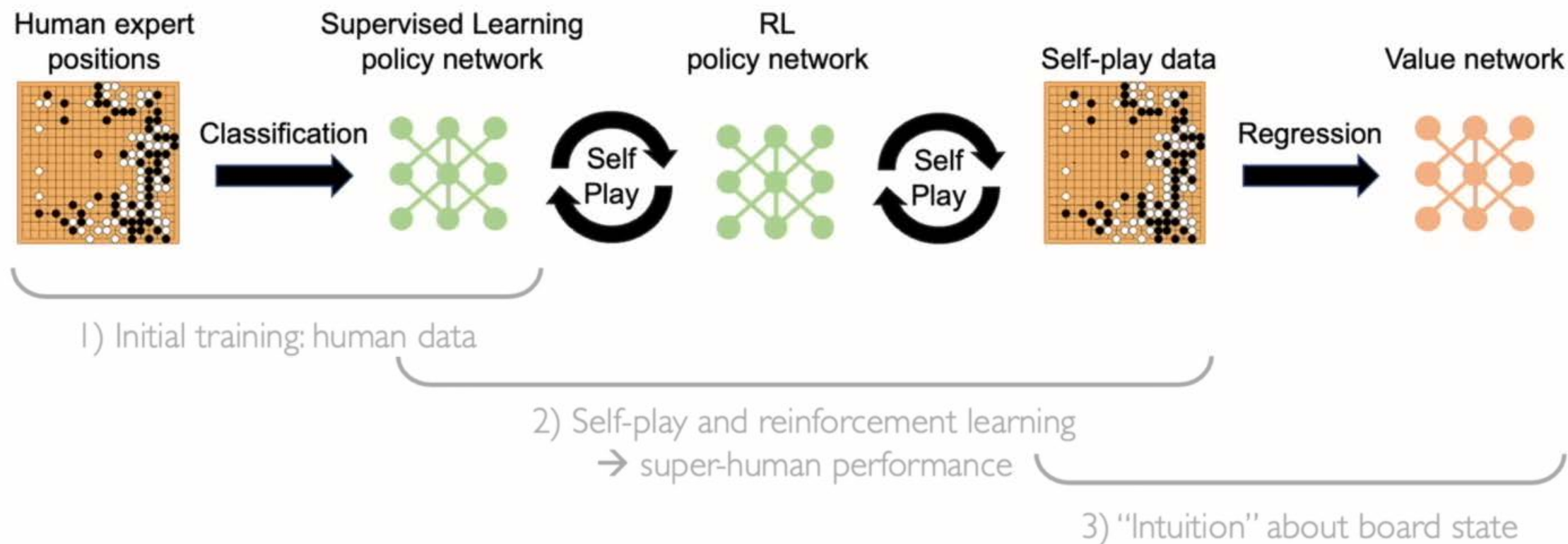
Aim: Get more board territory than your opponent.



Board Size $n \times n$	Positions 3^{n^2}	% Legal	Legal Positions
1×1	3	33.33%	1
2×2	81	70.37%	57
3×3	19,683	64.40%	12,675
4×4	43,046,721	56.49%	24,318,165
5×5	847,288,609,443	48.90%	414,295,148,741
9×9	$4.434264882 \times 10^{38}$	23.44%	$1.03919148791 \times 10^{38}$
13×13	$4.300233593 \times 10^{80}$	8.66%	$3.72497923077 \times 10^{79}$
19×19	$1.740896506 \times 10^{172}$	1.20%	$2.08168199382 \times 10^{170}$

Greater number of legal board positions than atoms in the universe.

AlphaGo Beats Top Human Player at Go



Deep Reinforcement Learning: Summary

Foundations

- Agents acting in environment
- State-action pairs $\rightarrow$ maximize future rewards
- Discounting



Q-Learning

- Q function: expected total reward given s, a
- Policy determined by selecting action that maximizes Q function



Policy Gradients

- Learn and optimize the policy directly
- Applicable to continuous action spaces



Lab 6

- Run dataset จาก Lab ที่แล้วทั้งหมด พร้อม evaluate ในระดับ Field Level, Page Level, Category Level : ส่งเป็นไฟล์ code ที่แสดงผลลัพธ์ ในกลุ่ม discord (งานเดี่ยว)
- ส่งภายใน 2 PM วันนี้